

COE 328 Lab 6 Report

Simple General-Purpose Processor Design

Name: Jaan Khan, Gurleen Kaur

Student ID: 501308663, 501245577

Due Date: Monday, December 1

Table of Contents

Introduction (3)

Component Analysis

- Latch A and B(3 - 4)
- Finite State Machine(4 - 5)
- 4 by 16 Decoder(5 - 6)

Arithmetic Logic Unit (Problem 1)(7 - 8)

Arithmetic Logic Unit (Problem 2)(9 - 10)

Arithmetic Logic Unit (Problem 3)(11 - 12)

Conclusion(12)

Appendix A.....(13 - 16)

Jaan's Photos

Gurleen's Photos

VHDL Code

Appendix B.....(17 - 19)

Jaan's Photos

Gurleen's Photos

VHDL Code

Appendix C(20 - 21)

Jaan's Photos

Gurleen's Photos

VHDL Code

Introduction

The purpose of this lab is to emulate a simple general-purpose processor through an FPGA board. The general processor unit (GPU) performs operations upon the input strings A and B. The strings A and B are chosen as the final 4 digits of the student ID, grouped into 2 and converted to binary:

- Jaan: 8663
 - $A = (86)_{10} = (01010110)_2$
 - $B = (63)_{10} = (00111111)_2$
- Gurleen: 5577
 - $A = (55)_{10} = (00110111)_2$
 - $B = (77)_{10} = (01001101)_2$

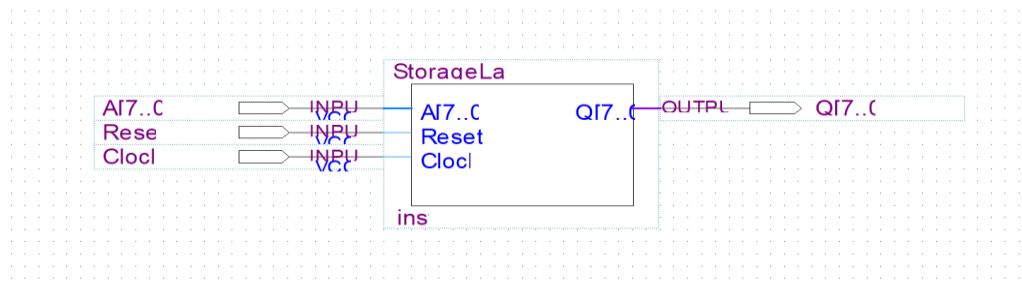
These two input vectors are held as storage in the two latches, which serve as memory registers. Upon the rising edge of the clock, the two latches will forward the input vectors A and B to the arithmetic logic unit (ALU). The rising edge of the clock will also sequence the finite state machine, operating as a Moore machine, which iterates through the student ID and advances the current state in the order: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 0$. The current state value will then input to the 4x16 decoder. This decoder (implemented through two 3x8 decoders), will output a 16-bit vector with only one bit on at a time. This output is called the OP-code (operational microcode). The microcode serves as the instructions given to the ALU to operate on the input strings A and B.

Finally, when the ALU performs an operation based off of the instruction fed by the OP-code, the result which is 8-bits will be broken into an upper and lower nibble, and both are converted to hexadecimal and outputted through seven segmentations. Additionally, the student ID sequenced through the FSM will also be outputted through one more seven segmentation HEX.

Component Analysis

Latch A and B

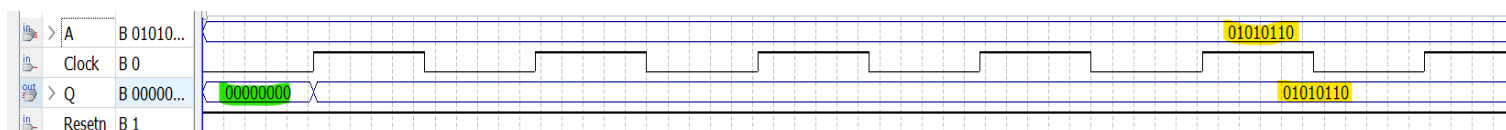
The purpose of the latch is simply to relay the input vectors A and B depending on the clock, and the condition of the reset flag. If the clock experiences a rising edge, then the input vectors are read, and they will be forwarded on the following rising edge. This will introduce a delay in the outputting of the data, and it will buffer the entire system by one clock cycle. This latency that the entire GPU will experience is due to the nature of the temporary register.



The truth table for the latches shows that the output will follow the input on every rising edge:

Input		Output
Clock	A	Q
0	0	X
0	1	X
1	0	0
1	1	1

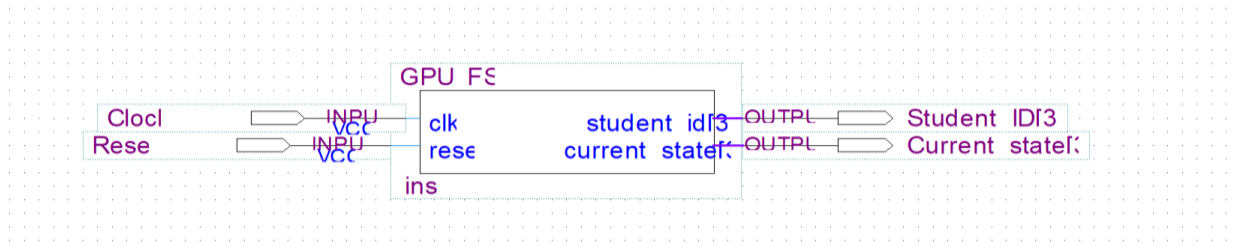
For the input vector $A = (01010110)_2$, the waveform of the latch will display that the vector is simply being forwarded after a delay due to latency:



This waveform shows the input and output vector A (yellow) after a delay (green).

Finite State Machine (FSM)

The Moore FSM sequences through states in the order $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 0$ where each current state corresponds to the digits of the student ID in sequential order. Therefore, upon the rising edge of the clock, the student ID will forward to a seven segmentation HEX and the current state will increment. Additionally, the reset flag will force the FSM to return to state 0, which represents the student ID digit 5. Typically, in a Moore machine, the data_in input doesn't necessarily serve a purpose because the machine will increment regardless so it was omitted in the design for the purpose of clarity.

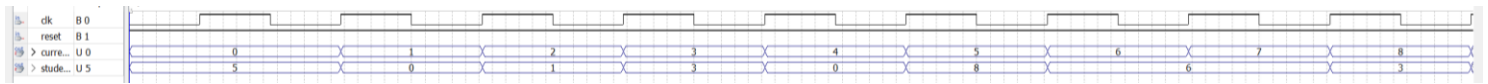


The FSM truth table can be shown as a state table with current state, next state, and the student ID output:

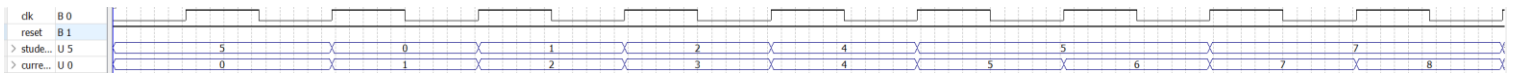
Present State	Next State	Output (Jaan)	Output (Gurleen)
s_0	s_1	5	5
s_1	s_2	0	0
s_2	s_3	1	1
s_3	s_4	3	2
s_4	s_5	0	4
s_5	s_6	8	5
s_6	s_7	6	5
s_7	s_8	6	7
s_8	s_0	3	7

The waveform of the FSM shows that each state is synchronized to a student ID digit:

Waveform for Jaan's student ID: 501308663



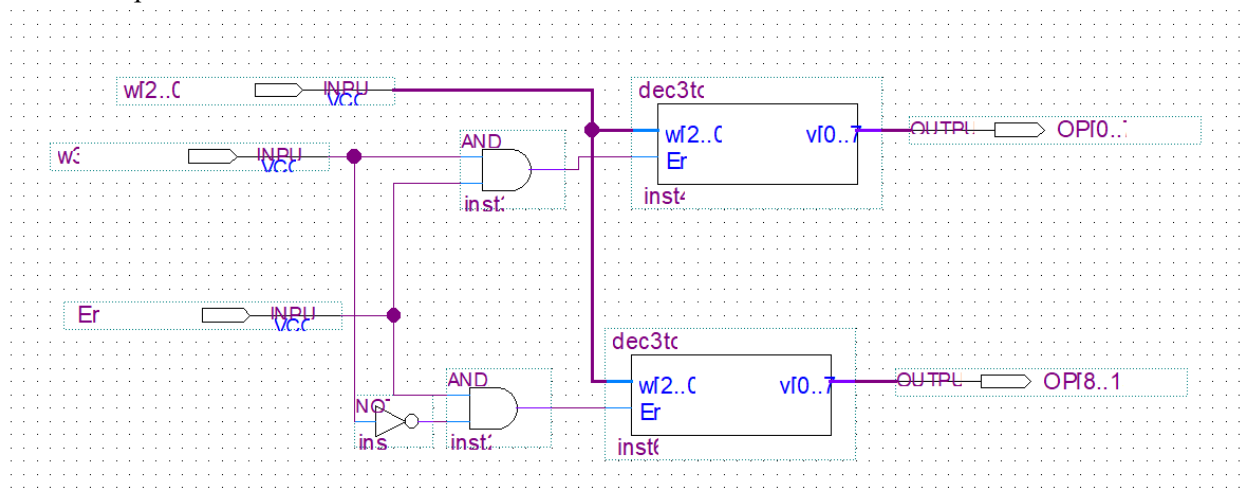
Waveform for Gurleen's student ID: 501245577



4x16 Decoder

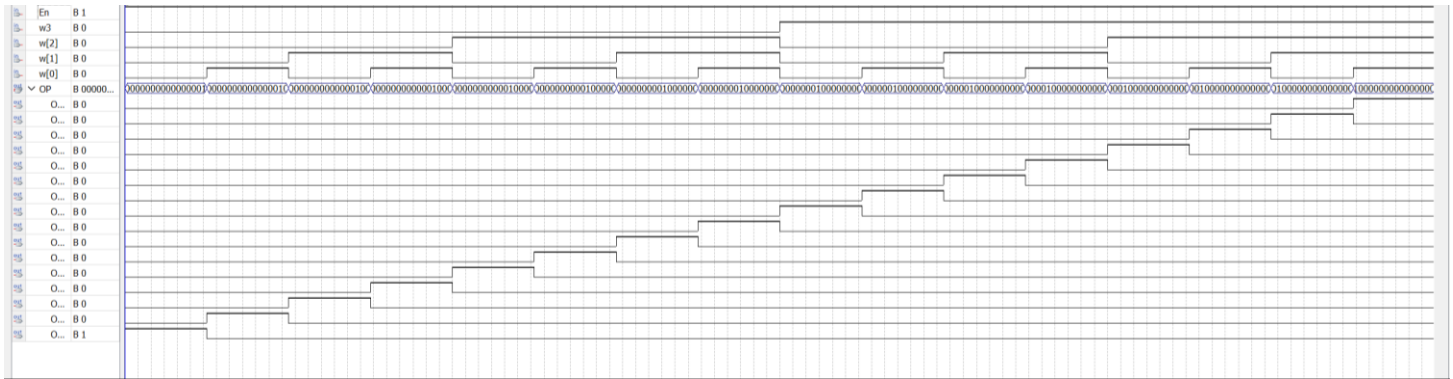
The purpose of the 4x16 decoder is to convert the current state that was outputted from the FSM, into functional microcode which instructs the ALU to perform operations. The 4x16 decoder takes in a 4 bit vector and outputs a 16-bit vector with only one bit active at a time. This 16 bit “microcode” will serve as instructions that the ALU will interpret and act upon. The 4x16 decoder was implemented using a combination of two 3x8 decoders and connecting them in a way such that only one is active at a time.

The 3x8 implementation of the 4x16 decoder looks as follows:



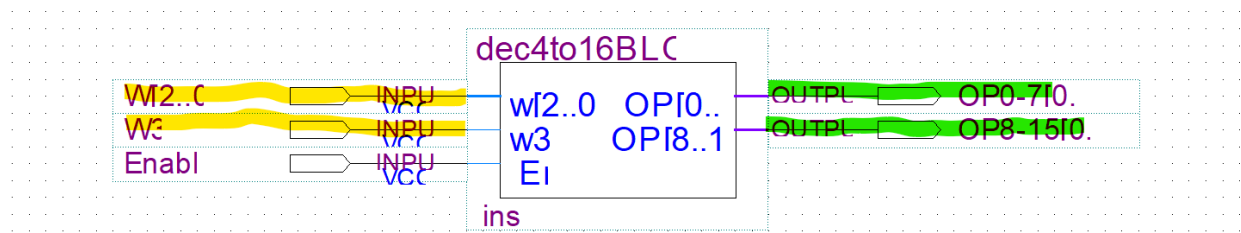
This circuit is designed to behave as a 4x16 decoder since the Enable and the w_3 output dictate which of the 3x8 decoders will turn on. When one turns on, the other will turn off, maintaining the “1 active bit” pattern of the microcode.

The waveform for the entire 4x16 decoder is as follows:



The waveform clearly displays that the OP code is transferring the “1-active bit” from the LSB to MSB, which shows that the 3x8 implementation of the 4x16 decoder is correct.

However, when creating the symbol for this 4x16 decoder, it posed an issue. Since w_3 is combined with the enable signal, it cannot be included in the input group $w[0..3]$. This means there must be 2 inputs $w[2..0]$ and separately w_3 (yellow). Similarly, the 3x8 decoders will output the OP code in two 8 bit vectors instead of one 16 bit vector. This means that there are two outputs $OP[0..7]$ and $OP[8..15]$ instead of just one output (green).



This problem was solved by splitting the one input bus wire into two wires ($w[2..0]$ and w_3) and joining the output bus wires into one wire ($OP[0..15]$), and appropriately naming the bus wires, so the data will split and join correctly. The truth table for the 4x16 decoder is as follows:

En	w_3	w_2	w_1	w_0	OP
1	0	0	0	0	0000000000000001
1	0	0	0	1	0000000000000010
1	0	0	1	0	0000000000000100
1	0	0	1	1	00000000000001000
1	0	1	0	0	00000000000010000
1	0	1	0	1	000000000000100000
1	0	1	1	0	0000000001000000
1	0	1	1	1	0000000010000000
1	1	0	0	0	0000000100000000
1	1	0	0	1	0000001000000000
1	1	0	1	0	0000010000000000
1	1	0	1	1	0000100000000000
1	1	1	0	0	0001000000000000
1	1	1	0	1	0010000000000000
1	1	1	1	0	0100000000000000
1	1	1	1	1	1000000000000000

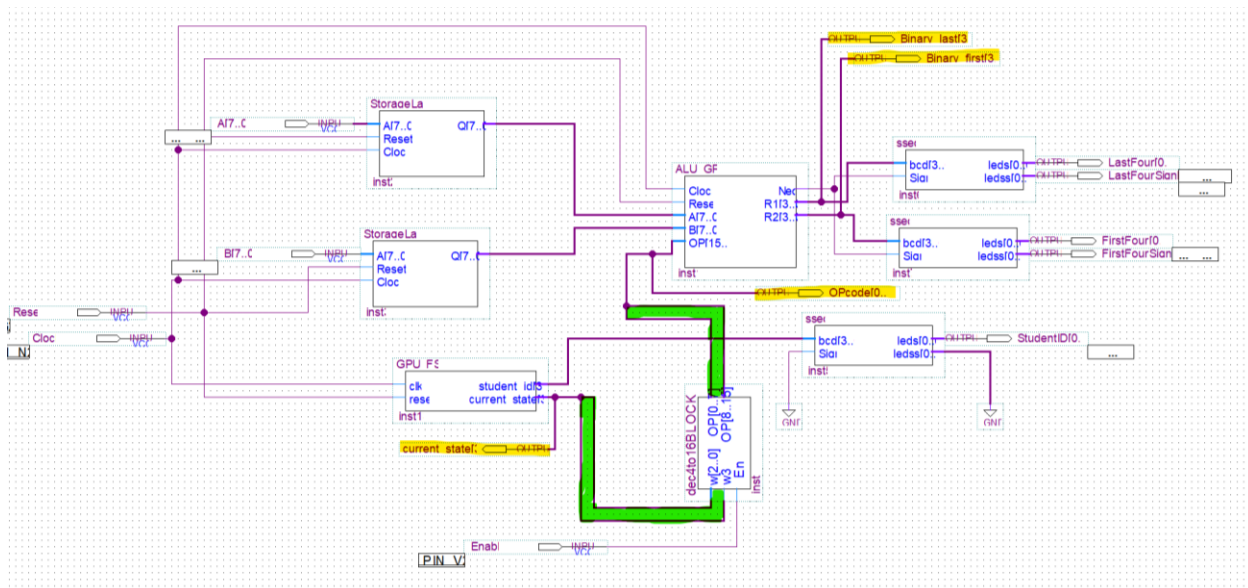
Arithmetic Logic Unit for Problem 1

The purpose of the ALU for problem 1 is to input the OP code generate by the decoder as it applies to the current state, and perform the associated operation on the input vectors A and B from the student ID. The list of instructions that the OP code can represent are as follows:

Microcode	Boolean Operation / Function
0000000000000001	sum(A, B)
0000000000000010	diff(A, B)
0000000000000100	$\overline{A}$
0000000000001000	$\overline{A \cdot B}$
0000000000010000	$\overline{A + B}$
0000000000100000	$A \cdot B$
0000000001000000	$A \oplus B$
0000000010000000	$A + B$
0000000100000000	$\overline{A \oplus B}$

When the ALU interprets the above microcode, it will perform the associated operation on the input vectors A and B.

The block diagram below shows each component of the GPU clearly and also shows the unique wiring (green) of the 4x16 decoder which allows for the 3x8 implementation.

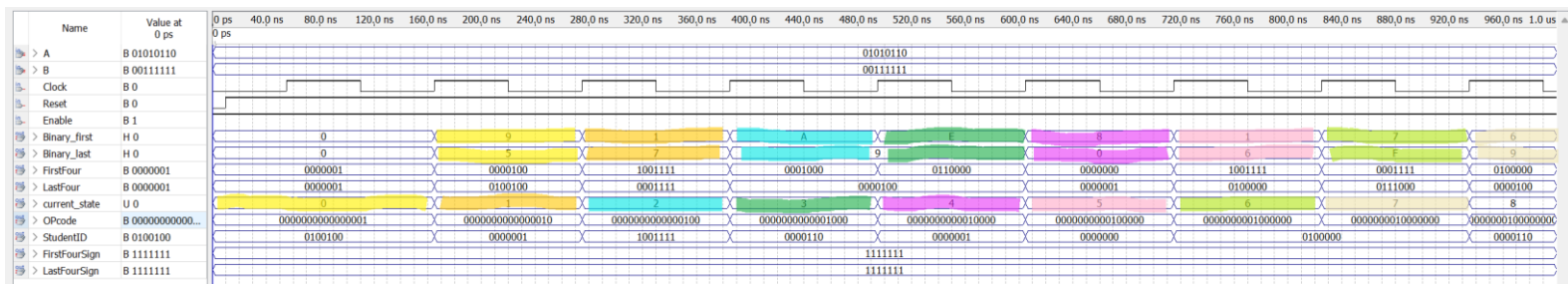


This block diagram also shows additional output signals (yellow) which were used to debug and verify the correct output. These debugging signals were simple ways to check if certain components like the decoder, ALU and sseg were interpreting the correct inputs and outputting the correct results.

The ALU will only operate on every rising edge of the clock, and the reset will force the ALU into its first output. The A and B inputs are simply the outputs from the latch, and the OP input is the instructional microcode delivered by the decoder. After performing operations upon the input vectors, the result will always be 8 bits and can include negative numbers. A negative number will raise the Neg output flag and will be displayed in the seven segmentation. The 8-bit output will be broken into an upper and lower nibble, and both these nibbles will be converted to hexadecimal and displayed through the seven segmentation. This will result in the following expected output based on the OP code generated due the current state:

Current State	ALU Output (Upper nibble, Lower Nibble)
0	9, 5
1	1, 7
2	A, 9
3	E, 9
4	8, 0
5	1, 6
6	6, 9
7	7, F
8	9, 6

The waveform for the ALU is as follows, where the associated outputs are color-coordinated:



As the waveform shows, the outputs are perfectly correct and align with the values listed in the table. However, the correct ALU output is delayed by one cycle relative to the corresponding current state. This delay is introduced due to the presence of the latch register, and the latency that it creates.

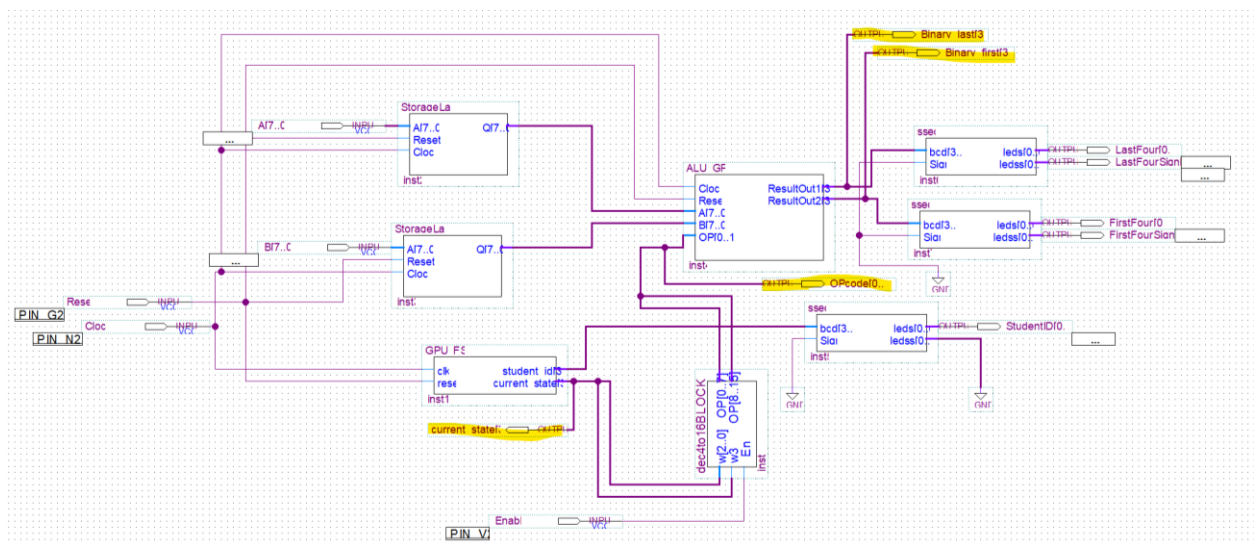
The photos of the FPGA board and the VHDL code for the functional ALU for problem 1 are attached in **Appendix A**

Arithmetic Logic Unit for Problem 2: Modification A)

The purpose of the ALU for problem 2 of this lab is to perform different, unique functions upon the input vectors A and B. The list of functions that the modified ALU can perform are as follows:

Function Number	OP code	Function
1	0000000000000001	Increment A by 2
2	0000000000000010	Shift B to right by two bits, input bit = 0 (SHR)
3	0000000000000100	Shift A to right by four bits, input bit = 1 (SHR)
4	0000000000001000	Find the smaller value of A and B and produce the results Min(A,B)
5	0000000000010000	Rotate A to right by two bits (ROR)
6	000000000100000	Invert the bit-significance order of B
7	000000001000000	Produce the result of XORing A and B
8	000000010000000	Produce the summation of A and B, then decrease it by 4
9	000000010000000	Produce all high bits on the output

The modified ALU core does not have the availability to produce a negative sign, so the negative output signal was deleted and the 'Sign' terminals of the seven segmentation were both grounded, to simplify the design. The modified ALU block diagram is shown below, with the additional debugging output signals to verify results (yellow):



As is shown in block diagram, the ALU now has only 2 outputs representing the upper and lower nibbles of the 8 bit result vector, since the sign can no longer be negative.

Overall, this GPU has 5 input signals and 3 output HEXs. The inputs are:

- Clock: Oscillating signal which controls the flow of the circuit on every rising edge
- Reset: Forces the FSM into its first state, which alters the ALU output
- Enable: 'On' switch for the decoder which is extremely important due to the 3x8 implementation of the 4x16 decoder
- A input vector: 8-bit input of 3rd and 4th last student ID digits
- B input vector: 8-bit input of 2nd last and last student ID digits

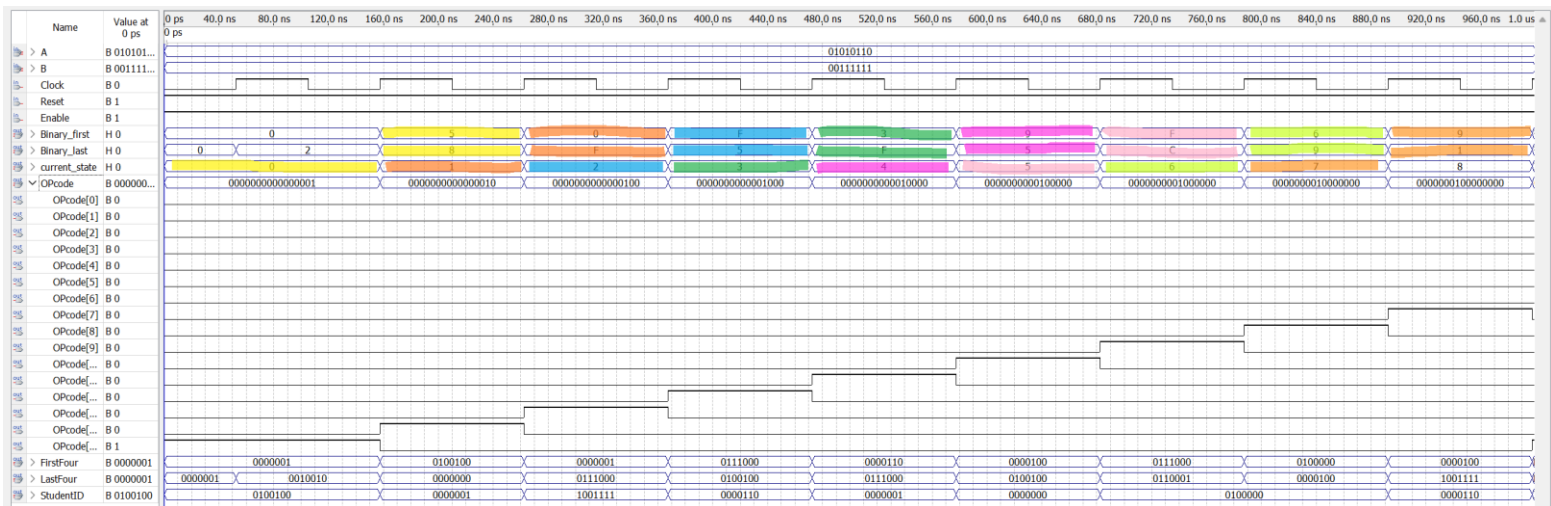
The outputs of the overall GPU are:

- FirstFour: Corresponds to the upper nibble of the ALUs output, which is displayed as hexadecimal through seven segmentation
- LastFour: Corresponds to the lower nibble of the ALUs output, which is displayed as hexadecimal through seven segmentation
- StudentID: Displays the current student ID being iterated, which corresponds to the current state as well

The expected outputs for the modified ALU are as follows:

Current State	Function	Result (Binary)	Result (Upper Nibble, Lower Nibble)
0	Increment A by 2	0101100	5, 8
1	SHR B 2 bits, Input = 0	00001111	0, F
2	SHR A 4 bits, Input = 1	11110101	F, 5
3	Min (A, B)	00111111	3, F
4	ROR A 2 bits	10010101	9, 5
5	Invert B	11111100	F, C
6	A XOR B	01101001	6, 9
7	A + B - 4	10010001	9, 1
8	High Output	11111111	F, F

The color-coded waveform is shown below, and the OP-code probe is also shown to progress in the correct pattern:



This waveform aligns perfectly with the expected values shown in the table, but again there is a delay between the output result and the associate current state. The delay is justified as the latency introduced through the use of the latch. Since the latch forwards the input vector on the following clock, everything is delayed by one clock cycle.

The photos of the FPGA board and the VHDL code for the functional ALU for problem 2 are attached in **Appendix B**

Arithmetic Logic Unit for Problem 3: Modification A)

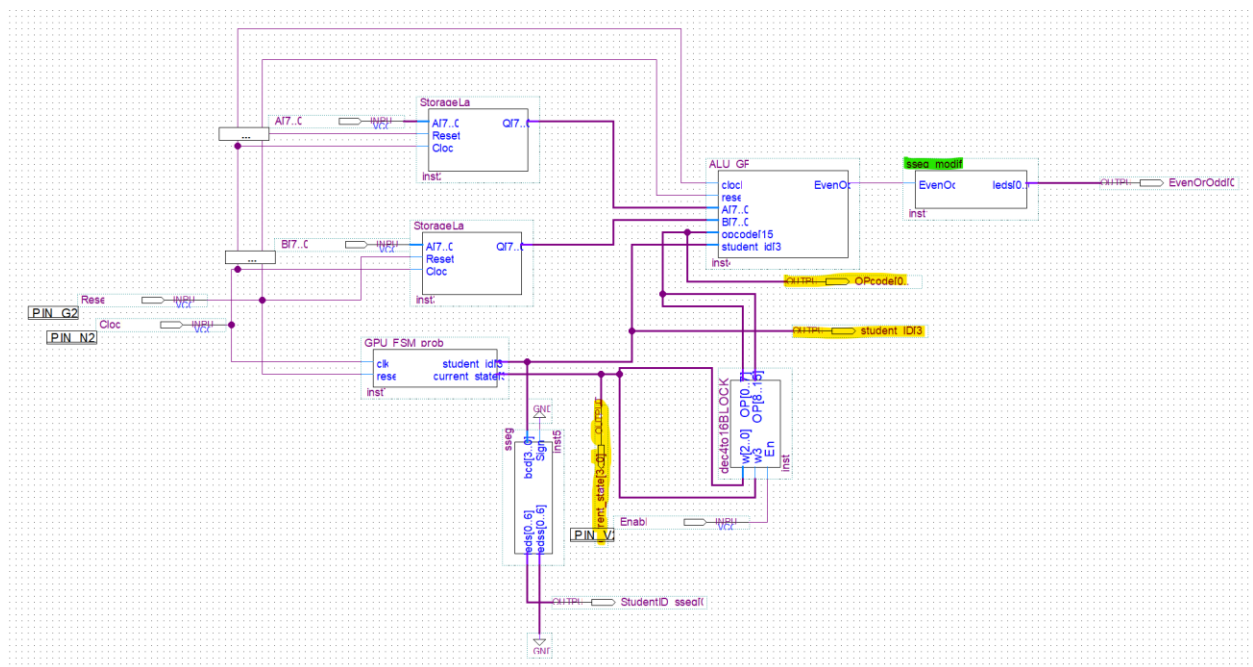
The purpose of the ALU for problem 3 of this lab is to perform checks on every digit of the student ID as the FSM cycles through it. Specifically, modification A states:

“For each microcode instruction, display 'y' if the FSM output (student_id) is odd and 'n' otherwise”

This goal was achieved through modifying both the ALU core and modifying the seven segmentation element. The original sseg element did not have the capability to output the letters ‘y’ and ‘n’, so a new simpler sseg was created. This modified sseg element will input a 1 bit signal, and output ‘y’ if the signal is high and ‘n’ if low. Additionally, the sign HEX is removed for design clarity since the output of a negative sign is not relevant.

In order to determine if the current student ID digit is even or odd, the ALU checks the last bit in the binary representation of the digit. If the last bit (which represents 1) is high, then that means the student ID must be odd since the remaining bits all represent even numbers (2, 4, 8, 16, 32...). This logic was used to determine if the current student ID digit was even or odd and then outputs a 1-bit signal (high if odd, low if even).

The block diagram of the modified GPU is as follows, with the modified sseg (green) and all the additional debugging verification probes (yellow):

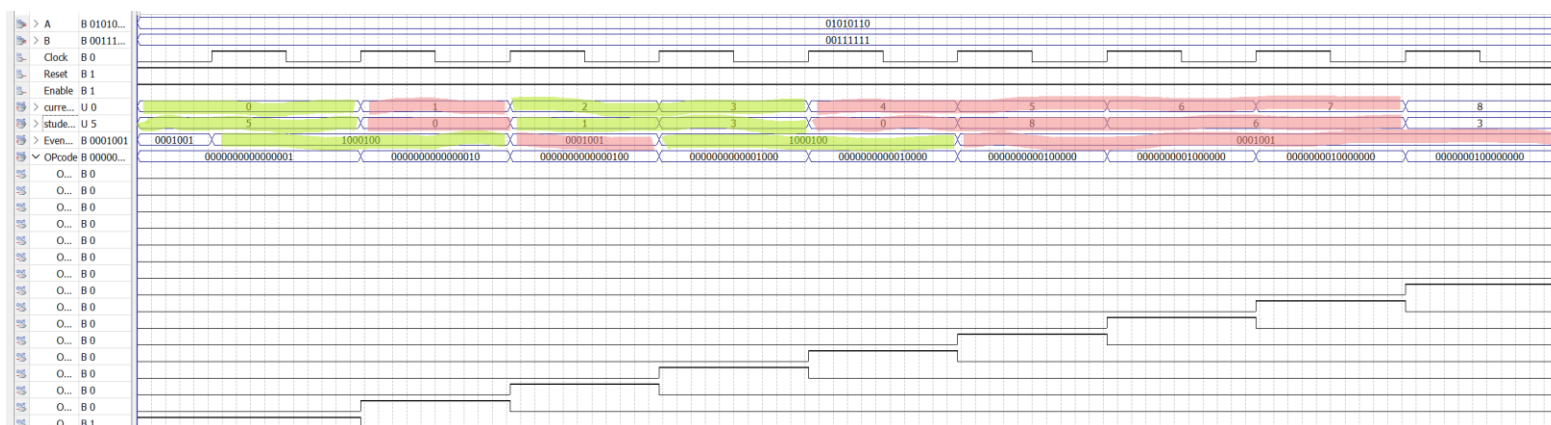


The overall inputs and outputs are relatively the same as the previous two iterations of the ALU, but the output is much simpler. There are two ALU outputs shown as hexadecimal on the seven segmentation HEX. The EvenOrOdd signal is the ‘y’ or ‘n’ output and the Student ID signal is the current digit of the student ID.

The expected output of the ALU core is as follows, where the sseg for Y is '1000100' and N is '0001001':

Current State	Student ID	Output (Y / N)
0	5	Y
1	0	N
2	1	Y
3	3	Y
4	0	N
5	8	N
6	6	N
7	6	N
8	3	Y

The color coded waveform is shown below with the OP-code progressing perfectly:



The waveform aligns with the table of expected output up to the buffer and delay caused by the latch. After considering the latency delay, the output can be justified to be acceptable.

The photos of the FGPA board and the VHDL code for the functional ALU for problem 3 are attached in **Appendix C**

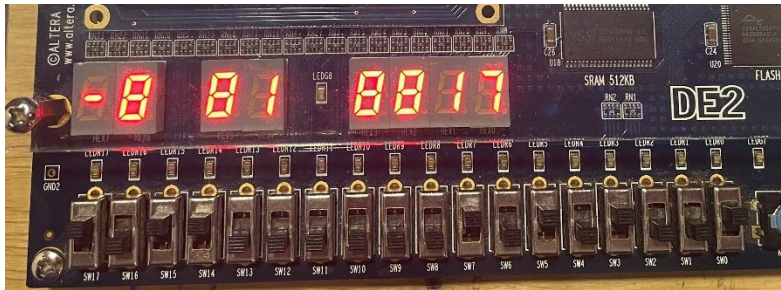
Conclusion

Ultimately, the general purpose processor was designed to satisfy all criteria, while keeping efficiency high and additional cost/complexity low. Errors were studied and debugged using output probes, and many sample waveforms were tried and tested. The specific objective of using a 3x8 implementation of the 4x16 decoder posed a challenge, but the problem was solved after researching and studying the output by using debugging probes. Overall, every result across the three ALUs met the objectives after considering the latency delay.

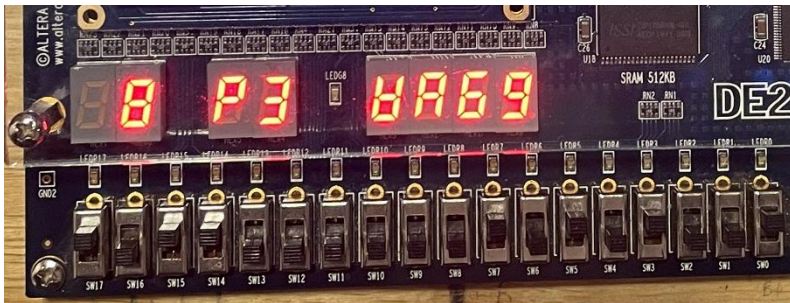
In conclusion, this lab served as a benchmark of understanding for the entire digital systems course. Since so much of the knowledge necessary to successfully design the general purpose processor was cumulative knowledge, this lab served a purpose of summarizing all the previously learned concepts in lectures and previous labs altogether.

Appendix A

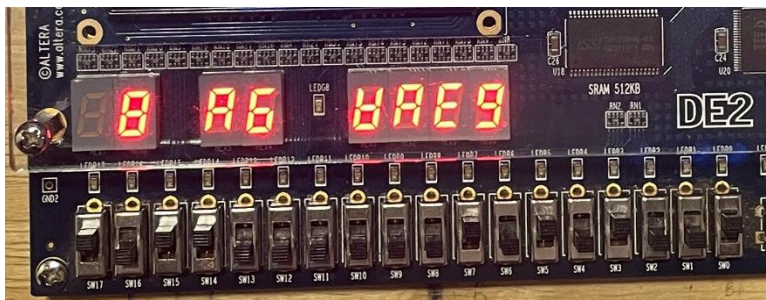
Jaan (501308663) photos for Problem 1:



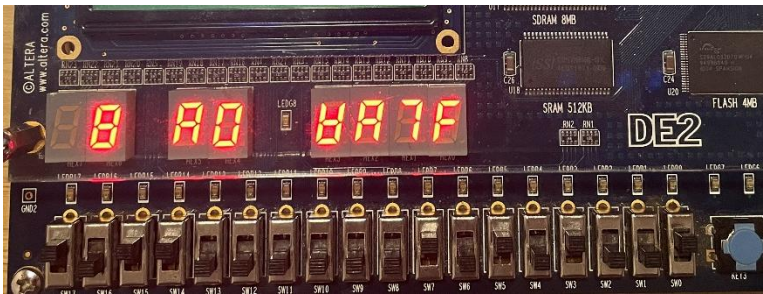
Output: 1 7



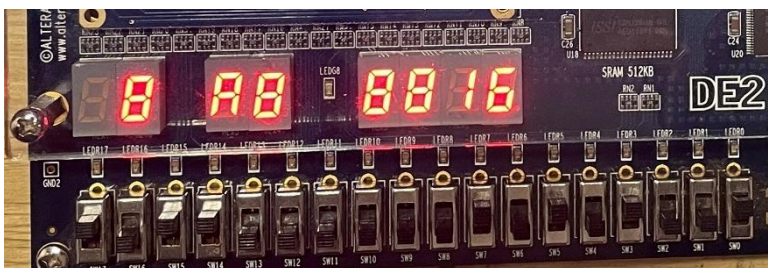
Output: 6 9



Output: E 9



Output: 7 F



Output: 1 6

Gurleen (501245577) photos for Problem 1:



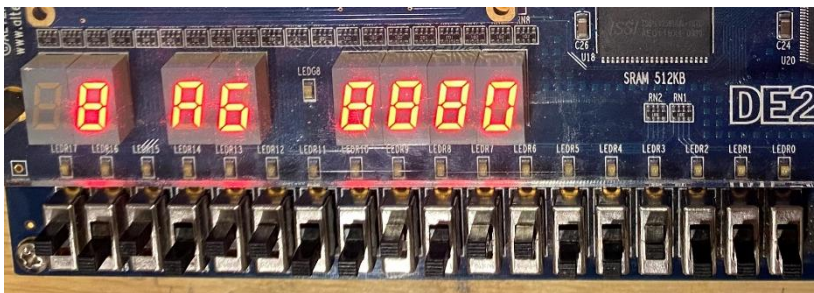
Output: 7 F



Output: F A



Output: 8 5



Output: 8 0



Output: 7 A

VHDL Code for ALU Problem 1:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;
entity ALU_GPU is
    port(
        Clock : in std_logic;
        Reset : in std_logic; -- ACTIVE LOW RESET
        A, B : in unsigned(7 downto 0);
        OP : in unsigned(15 downto 0);
        Neg: out std_logic;
        R1 : out unsigned(3 downto 0);
        R2 : out unsigned(3 downto 0)
    );
end ALU_GPU;
architecture calculation of ALU_GPU is
    signal Reg1, Reg2, Result : unsigned(7 downto 0) := (others => '0');
begin
    Reg1 <= A;
    Reg2 <= B;

    process(Clock, OP, reset)
    begin
        if Reset = '0' then
            -- ACTIVE-LOW RESET BEHAVIOR
            Result <= (others => '0');
            Neg <= '0';
        elsif rising_edge(Clock) then
            Neg <= '0'; -- default
            case OP is
                WHEN "0000000000000001" => -- ADD
```

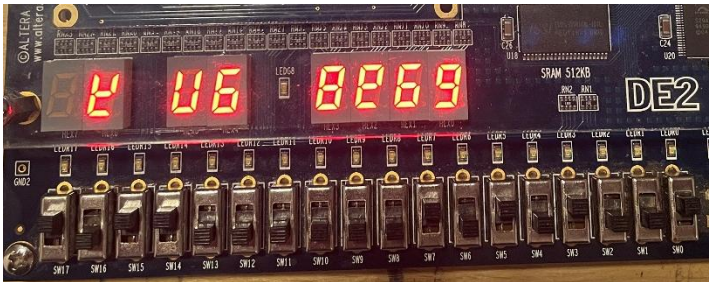
```

    Result <= Reg1 + Reg2;
WHEN "0000000000000010" => -- SUB
    if Reg1 >= Reg2 then
        Result <= Reg1 - Reg2;
        Neg <= '0';
    else
        Result <= Reg2 - Reg1;
        Neg <= '1';
    end if;
WHEN "0000000000000100" => -- NOT A
    Result <= not Reg1;
WHEN "0000000000001000" => -- NAND
    Result <= not (Reg1 and Reg2);
WHEN "0000000000010000" => -- NOR
    Result <= not (Reg1 or Reg2);
WHEN "0000000000100000" => -- AND
    Result <= Reg1 and Reg2;
WHEN "0000000001000000" => -- OR
    Result <= Reg1 or Reg2;
WHEN "0000000010000000" => -- XOR
    Result <= Reg1 xor Reg2;
WHEN "0000000100000000" => -- XNOR
    Result <= Reg1 xnor Reg2;
WHEN OTHERS =>
    Result <= "11111111";
end case;
end if;
end process;
R1 <= Result(3 downto 0);
R2 <= Result(7 downto 4);
end calculation;

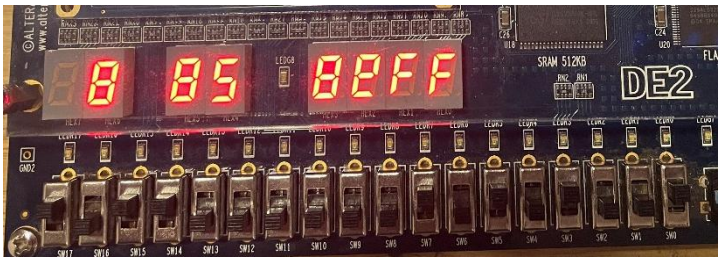
```

Appendix B

Jaana (501308663) photos for Problem 2:

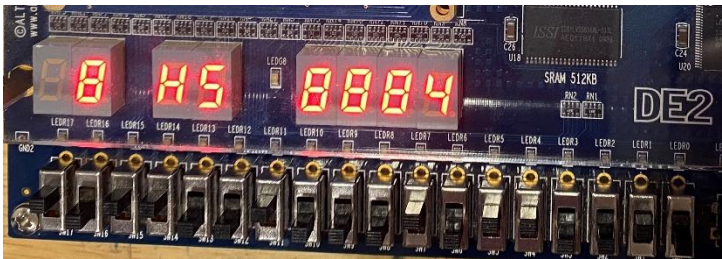


Output: 6 9



Output: F F

Gurleen (501245577) photos for Problem 2:



Output: 8 4



Output: 9 6



Output: 3 7

VHDL Code for ALU Problem 2:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity ALU_GPU is
    port(
        Clock : in std_logic;
        Reset : in std_logic;
        A, B : in unsigned(7 downto 0);
        OP : in unsigned(0 to 15);
        ResultOut1 : out unsigned(3 downto 0);
        ResultOut2 : out unsigned(3 downto 0)
    );
end entity;
architecture Behavioral of ALU_GPU is
    signal Result : unsigned(7 downto 0) := (others => '0');
begin
    process(Clock)
    begin
        if rising_edge(Clock) then
            if Reset = '0' then
                Result <= (others => '0');
            else
                -- Default result (optional)
                Result <= (others => '0');
                case OP is
                    -- Function 1: Increment A by 2
                    when "0000000000000001" =>
                        Result <= A + 2;
                    -- Function 2: B >> 2 fill 0
                    when "0000000000000010" =>
                        Result <= "00" & B(7 downto 2);
                    -- Function 3: A >> 4 fill 1
```

```

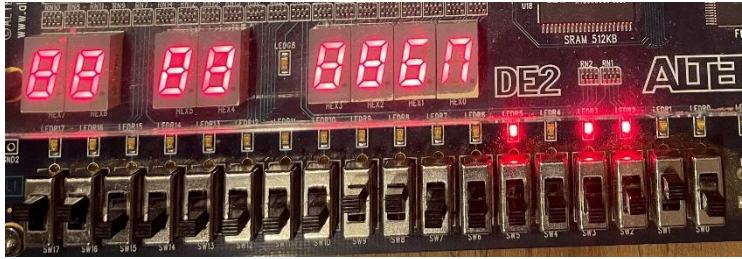
when "0000000000000100" =>
    Result <= "1111" & A(7 downto 4);
-- Function 4: min(A, B)
when "0000000000001000" =>
    if A <= B then Result <= A;
    else Result <= B;
    end if;
-- Function 5: ROR A by 2
when "0000000000010000" =>
    Result <= A(1 downto 0) & A(7 downto 2);
-- Function 6: reverse B
when "0000000000100000" =>
    Result <= B(0) & B(1) & B(2) & B(3) &
        B(4) & B(5) & B(6) & B(7);
-- Function 7: A XOR B
when "0000000001000000" =>
    Result <= A xor B;
-- Function 8: A + B - 4
when "0000000010000000" =>
    Result <= (A + B) - 4;
-- Function 9: All 1s
when "0000000100000000" =>
    Result <= (others => '1');
when others =>
    Result <= (others => '0');
end case;
end if;
end if;
end process;

ResultOut1 <= Result(3 downto 0);
ResultOut2 <= Result(7 downto 4);
end architecture;

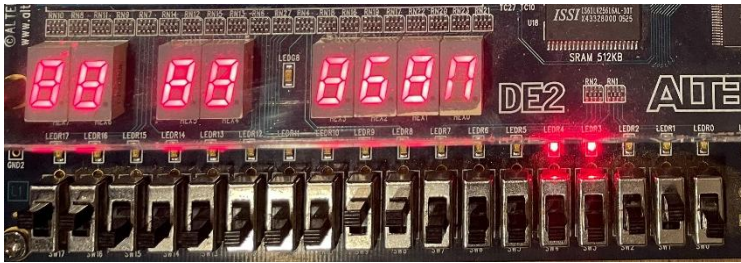
```

Appendix C

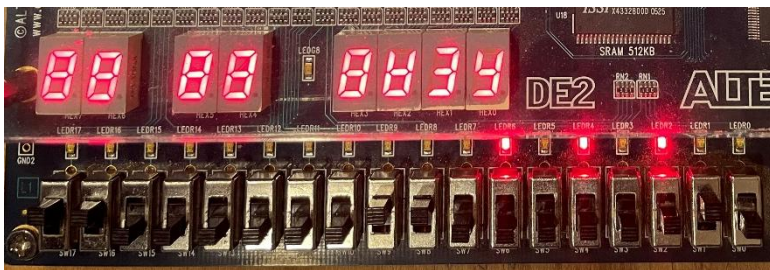
Jaan (501308663) photos for Problem 3:



Output: 6 N



Output: 8 N

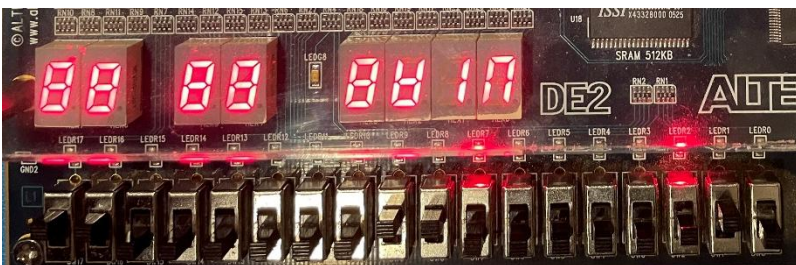


Output: 3 Y

Gurleen (501245577) photos for Problem 3:



Output: 5 Y



Output: 1 N

VHDL Code for ALU Problem 3:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ALU_GPU is
    port(
        clock    : in std_logic;
        reset    : in std_logic;
        A        : in unsigned(7 downto 0);
        B        : in unsigned(7 downto 0);
        opcode    : in std_logic_vector(15 downto 0);
        student_id : in unsigned(3 downto 0);
        EvenOdd    : out std_logic -- 1 = YES, 0 = NO
    );
end entity;

architecture Behavioral of ALU_GPU is
    signal temp : unsigned(3 downto 0) := (others => '0');
begin
    process(clock)
    begin
        if rising_edge(clock) then
            if reset = '0' then
                temp <= (others => '0');
                EvenOdd <= '0';
            else
                -- YES = student_id is odd
                if student_id(0) = '1' then
                    EvenOdd <= '1'; -- YES
                else
                    EvenOdd <= '0'; -- NO
                end if;
            end if;
        end if;
    end process;
end architecture Behavioral;
```